PERSONAL EXPENSE TRACKER & ANALYSIS SYSTEM

A PROJECT REPORT

SUBMITTED BY

ROLL NUMBER	REGISTRATION NUMBER	STUDENT CODE	STUDENT NAME
26102410029	**26102000151**	**BWU/MCA/26/033**	**RITAM BERA**

in partial fulfillment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



NOVEMBER 2026

PERSONAL EXPENSE TRACKER & ANALYSIS SYSTEM

A PROJECT REPORT

SUBMITTED BY

ROLL NUMBER	REGISTRATION NUMBER	STUDENT CODE	STUDENT NAME
26102410029	**26102000151**	**BWU/MCA/26/033**	**RITAM BERA**

in partial fulfillment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ABSTRACT

The Personal Expense Tracker & Analysis System is a Python-based web application developed to record daily personal expenses and examine spending patterns through an interactive dashboard. The application allows a user to enter an expense date, select a category, specify an amount and provide a short description. Expense records are stored in a CSV file so that they remain available between application sessions.

The system is implemented using Python and Streamlit. Pandas is used to read, clean, filter, sort and summarize expense data, while Matplotlib generates monthly, category-wise, daily trend and distribution charts. The dashboard displays total, average, highest and lowest expense values. Users can filter records by category, sort them by date or amount, inspect the highest and lowest expense records, and download the expense data as a CSV file.

The application also provides an optional receipt or expense-photo upload feature for JPG, JPEG and PNG files. Uploaded images are saved in a separate expense_photos folder. The supplied source package contains app.py, expenses.csv and requirements.txt, together with a bundled virtual environment. The CSV supplied with the project contains six sample expense records dated September and October 2026. Based on those records, the total expenditure is ₹2,090.00, the average expense is ₹348.33, the highest expense is ₹800.00 and the lowest expense is ₹120.00.

The project demonstrates Python functions, conditional statements, data collections through Pandas DataFrames, CSV file handling, input validation, data grouping, sorting and graphical visualization. It is intended for individual academic demonstration and can be extended with editing and deletion facilities, date-range filters, user authentication and a database in a future version.

TABLE OF CONTENTS

Chapter	Title	Page No.
	ABSTRACT	3
	LIST OF TABLES	6
	LIST OF FIGURES	7
1.	**INTRODUCTION**	8
	1.1 Background	8
	1.2 Problem Statement	8
	1.3 Scope of the Project	8
	1.4 Main Modules	9
2.	**OBJECTIVE**	10
	2.1 Primary Objective	10
	2.2 Secondary Objectives	10
	2.3 Functional Objectives	10
	2.4 Technical Objectives	10
	2.5 User-centric Objectives	11
	2.6 Long-term Objectives	11
3.	**PLANNING**	12
	3.1 Need for Planning	12
	3.2 Project Development Methodology	12
	3.3 Work Breakdown Structure	12
	3.4 Project Scheduling	13
	3.5 Module Identification	13
	3.6 Resource Planning	13
	3.7 Risk Analysis and Mitigation	14
4.	**REQUIREMENT ANALYSIS**	15
	4.1 User Requirements	15
	4.2 Functional Requirements	15
	4.3 Non-Functional Requirements	16
	4.4 System Requirements	16
	4.5 Data Requirements	17
	4.6 Existing System and Comparison	17
	4.7 Constraints and Assumptions	17
5.	**SYSTEM FLOW**	18

	5.1 Data Flow Diagram – Level 0	18
	5.2 Data Flow Diagram – Level 1	18
	5.3 Use Case Diagram	19
	5.4 Entity–Relationship Diagram	19
	5.5 Activity Diagram	20
	5.5 Flowchart	20
6.	**PROPOSED DESIGN**	21
	6.1 Design Approach	21
	6.2 System Architecture Design	21
	6.3 User Interface Planning	22
	6.4 Data / Module Design	22
	6.5 Design Considerations	22
7.	**EXPERIMENTAL RESULT**	23
	7.1 Experimental Setup	23
	7.2 Expense Entry Results	23
	7.3 Summary and Record Management Results	24
	7.4 Analytical Results	24
	7.5 Graphical Analysis Results	24-26
	7.6 Testing Results	27
8.	**FUTURE SCOPE**	28
9.	**CONCLUSION**	29
	APPENDICES	30
	APPENDIX A: APPLICATION INTERFACE	30-31
	APPENDIX B: SAMPLE SOURCE CODE	32
	APPENDIX C: CSV DATA STRUCTURE	33
	APPENDIX D: TEST CASES	34
	REFERENCES	35

LIST OF TABLES

Table No.	Title	Page No.
Table 1.4	Main modules of the Personal Expense Tracker	9
Table 3.2	Project development methodology	12
Table 3.4	Project schedule	13
Table 3.6	Resource planning	13
Table 3.7	Risk analysis and mitigation	14
Table 4.2	Functional requirements	15
Table 4.3	Non-functional requirements	16
Table 4.4	System requirements	16
Table 4.5	CSV data structure	17
Table 4.6	Existing system and comparison	17
Table 6.4	Data / module design	22
Table 7.3	Sample dataset summary	23
Table 7.4	Project test cases	24
Table 7.6	Testing Results	27
Table C.1	Expense CSV field description	26

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 5.1	Data Flow Diagram – Level 0	18
Figure 5.2	Data Flow Diagram – Level 1	18
Figure 5.3	Use Case Diagram	19
Figure 5.4	Entity–Relationship Diagram	19
Figure 5.5	Activity Diagram	20
Figure 5.6	Expense Entry Flowchart	20
Figure 6.1	System Architecture	21
Figure 7.1	Monthly Expenditure Chart	24
Figure 7.2	Category-wise Expenditure Chart	25
Figure 7.3	Expense Distribution Pie Chart	25
Figure 7.4	Daily Expense Trend	26
Figure 7.5	Expense Amount Histogram	26

CHAPTER 1

INTRODUCTION

Managing personal expenses is an important activity for understanding how money is used in daily life. Small payments for food, travel, education, shopping and entertainment can accumulate over time. When expenses are recorded manually in notebooks or separate files, it may become difficult to calculate totals, compare categories and identify spending trends.

The Personal Expense Tracker & Analysis System provides a simple browser-based dashboard for recording expenses and reviewing the resulting data. A user enters each expense through a form. The application saves the record in a CSV file and uses the saved data to display summary values, a sortable expense table and graphical analysis.

1.1 Background

The project is developed in Python using the Streamlit framework. Streamlit provides the browser-based interface and form controls. Pandas manages tabular expense data, and Matplotlib creates charts. The application is launched locally using the Streamlit command and reads the expenses.csv file located beside app.py.

1.2 Problem Statement

The project addresses the need for a straightforward tool to record personal expenses and analyze spending without maintaining separate manual calculations. The system should accept valid expense entries, preserve them in a reusable file, provide useful summary statistics, support category filtering and sorting, and present spending patterns through charts.

1.3 Scope of the Project

- Use six predefined categories: Food, Travel, Education, Shopping, Entertainment and Other.
- Display total, average, highest and lowest expense metrics.
- Show highest and lowest expense details when records exist.
- Analyze monthly and category-wise expenditure.
- Upload and display a JPG, JPEG or PNG receipt/photo, saving it in expense_photos.

1.4 Main Modules

Module	Description
Expense Entry	Collects date, category, amount and description through a Streamlit form.
Data Loading and Storage	Reads expenses.csv, ensures expected columns exist and writes updated records.
Summary Calculation	Calculates total, average, maximum and minimum expense amounts.
Record Management View	Displays expense rows with category filtering and four sorting options.
Spending Analysis	Groups data by month, category and date to calculate spending totals.
Visualization	Creates monthly bar, category bar, pie, daily line and histogram charts.
CSV Download	Provides the current expense data as a downloadable CSV file.
Receipt / Photo Upload	Accepts JPG, JPEG and PNG images and saves them in a separate folder.

CHAPTER 2

OBJECTIVE

2.1 Primary Objective

To develop a simple Python-based expense tracking application that allows a user to record daily expenses, store them in a CSV file and understand spending patterns through summary calculations and graphical analysis.

2.2 Secondary Objectives

- Reduce manual effort in maintaining daily expense records.
- Organize expenses using a fixed set of meaningful categories.
- Make expense records easier to search through category filtering and sorting.
- Provide quick insight into total, average, highest and lowest spending.
- Present monthly, category-wise and daily expenditure in graphical form.
- Allow users to retain a downloadable copy of their expense data.

2.3 Functional Objectives

- Accept the expense date, category, amount and description.
- Reject an amount that is not greater than zero.
- Warn the user when the description is blank.
- Append valid expense details to expenses.csv.
- Display all available expense records in a table.
- Filter records by a selected category and sort by date or amount.
- Calculate and display summary metrics and highest/lowest expense details.
- Generate charts and support CSV download and image upload.

2.4 Technical Objectives

- Use Python as the main programming language.
- Use Streamlit for page configuration, forms, metrics, tabs and file upload.
- Use Pandas for CSV reading, writing, data cleaning, grouping and sorting.
- Use Matplotlib for bar, pie, line and histogram visualizations.
- Use pathlib to construct project-relative paths for the CSV and photo directory.
- Keep the application simple enough to run locally for academic demonstration.

2.5 User-centric Objectives

The interface should provide clearly labelled input fields, category choices, confirmation messages and readable summary metrics. The expense table should allow the user to narrow the displayed records by category and choose a useful sorting order.

2.6 Long-term Objectives

Future versions may introduce editing and deletion of records, user authentication, date-range filtering, recurring expenses, budget limits, database storage, automatic receipt-to-expense linking and downloadable PDF reports.

CHAPTER 3

PLANNING

3.1 Need for Planning

Planning is required to identify the data fields, define the expense entry process, select the required Python libraries and decide how the records will be stored and analyzed. It also helps keep the project within a manageable academic scope.

3.2 Project Development Methodology

The project follows a simple incremental development approach. First, the expense fields and categories are identified. Next, CSV loading and saving functions are implemented. The input form and validation are added, followed by summary metrics, table filtering, sorting, analysis charts, download and photo upload.

Phase	Activity	Output
1. Requirement study	Identify expense fields, categories and user tasks.	Requirement list
2. Design	Plan data columns, application sections and analysis outputs.	Application design
3. Implementation	Develop CSV functions, form handling and calculations.	Working application
4. Visualization	Create monthly, category, daily and distribution charts.	Analytical charts
5. Testing	Check valid and invalid inputs, file operations and analysis.	Test observations
6. Documentation	Prepare diagrams, tables, results and report.	Project report

3.3 Work Breakdown Structure (WBS)

- Project initiation and requirement collection.
- Definition of expense categories and CSV columns.
- Development of CSV loading and saving functions.
- Design of the Streamlit expense entry form.
- Implementation of summary metrics and record table.
- Development of filtering, sorting and analytical charts.
- Implementation of CSV download and receipt/photo upload.
- Testing, diagram preparation and documentation.

3.4 Project Scheduling

Activity	Week 1	Week 2	Week 3	Week 4
Requirement and design	✓			
CSV handling and entry form	✓	✓		
Summary and record table		✓	✓	
Charts and analysis			✓	
Testing and documentation			✓	✓

3.5 Module Identification

The source code is organized in a single main Python file, app.py. The file contains page configuration, custom CSS, CSV loading and saving functions, the expense entry form, calculations, record display, analysis tabs, file download and image upload. The CSV data is stored separately in expenses.csv.

3.6 Resource Planning

Resource	Purpose
Python 3.9 or later	Main programming language and runtime; the supplied virtual environment metadata indicates Python 3.9.
Visual Studio Code	Code editing and project management.
Streamlit	Browser interface, forms, metrics, tables, tabs and upload controls.
Pandas	CSV operations and tabular data analysis.
Matplotlib	Graphical representation of expense data.
expenses.csv	Persistent storage for expense records.
Web browser	Access the locally running Streamlit application.

3.7 Risk Analysis and Mitigation

Risk	Mitigation / current handling
Invalid or missing amount	The form sets a minimum amount of ₹0.01 and checks that the submitted amount is greater than zero.
Blank description	The application checks description.strip() and displays a warning when it is empty.
Missing CSV file	The loader creates an empty CSV with the expected columns when the file does not exist.
Unexpected CSV read error	The loader catches a read exception and returns an empty DataFrame with the expected columns.
Incorrect data types	Dates are converted with pandas.to_datetime and amounts with pandas.to_numeric.
Photo file handling	Only JPG, JPEG and PNG file types are accepted by the uploader.

CHAPTER 4

REQUIREMENT ANALYSIS

4.1 User Requirements

- The user needs to enter the date, category, amount and description of an expense.
- The user needs to view saved expense records in a table.
- The user needs to filter records by category and sort records by date or amount.
- The user needs to see summary values and identify the highest and lowest expense.
- The user needs to review monthly and category-wise expenditure using charts.
- The user needs to download the CSV data and optionally upload a receipt or expense photo.

4.2 Functional Requirements

ID	Functional requirement
FR-01	The system shall display an expense entry form with date, category, amount and description fields.
FR-02	The system shall provide six predefined expense categories.
FR-03	The system shall prevent submission of an amount that is zero or negative.
FR-04	The system shall warn the user when the description is blank.
FR-05	The system shall save valid expense records in expenses.csv.
FR-06	The system shall load and display the stored expense records.
FR-07	The system shall calculate total, average, highest and lowest amounts.
FR-08	The system shall support category filtering and date/amount sorting.
FR-09	The system shall display monthly and category-wise expenditure analysis.
FR-10	The system shall generate daily trend and amount distribution charts.
FR-11	The system shall provide CSV download and receipt/photo upload.

4.3 Non-Functional Requirements

Attribute	Description
Usability	Clear labels, form controls, metrics and analysis tabs.
Reliability	CSV loading ensures expected columns and converts date and amount values to suitable types.
Maintainability	The application is contained in a single readable Python file with named sections and functions.
Performance	Suitable for small personal expense datasets stored locally in CSV format.
Portability	Can run on a system with Python and the packages listed in requirements.txt.
Data persistence	Expense rows are saved to a CSV file; the uploaded photos are saved separately.

4.4 System Requirements

Category	Requirement
Hardware	Desktop or laptop with sufficient memory, keyboard and mouse.
Operating system	Windows, Linux or macOS.
Python	Python 3.9 or later is consistent with the supplied project environment; requirements specify Streamlit, Pandas and Matplotlib.
Editor	Visual Studio Code recommended.
Browser	Modern web browser.
Libraries	Streamlit, Pandas and Matplotlib.
Application server	Streamlit local application server.

4.5 Data Requirements

The supplied application uses one CSV file named expenses.csv. The file contains four columns. The source code expects the same four columns and creates missing columns when loading an existing file.

Field	Data type / use	Description
Date	Date	Date on which the expense occurred.
Category	Text	One of Food, Travel, Education, Shopping, Entertainment or Other.
Amount	Numeric	Expense value in Indian rupees.
Description	Text	Short explanation of the expense.

4.6 Existing System and Comparison

Manual approach	Proposed application
Expenses written in notebooks or separate sheets.	Expense entries are collected through one browser form.
Totals and averages calculated manually.	Summary metrics are calculated using Pandas.
Category comparisons require manual grouping.	Category filter and category-wise chart are available.
Monthly trends require manual chart preparation.	Monthly expenditure chart is generated from the data.
Records are not always easy to share.	CSV download is provided.

4.7 Constraints and Assumptions

The current version stores expenses in a local CSV file and does not include login, multi-user separation, editing or deletion of saved records. The photo uploader saves files separately in expense_photos and does not automatically associate a photo with a particular expense row. The project assumes that the application folder is writable and that the required Python libraries are installed.

CHAPTER 5

SYSTEM FLOW

5.1 Data Flow Diagram – Level 0

The Level 0 DFD presents the user as the external entity. The application receives expense details and photo uploads. Expense data is stored in expenses.csv, while uploaded images are saved separately in the expense_photos folder. The application reads stored expense data to calculate summaries and prepare charts.

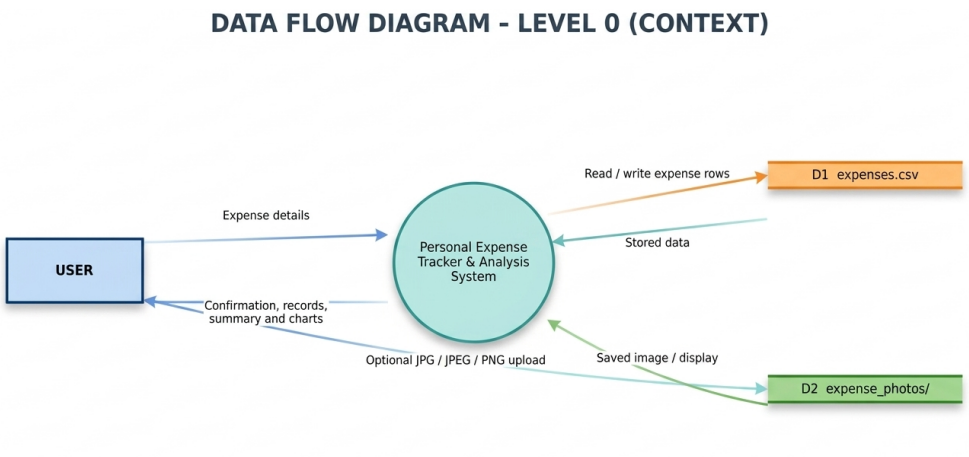


Figure 5.1 – Data Flow Diagram Level 0

5.2 Data Flow Diagram – Level 1

The Level 1 DFD expands the expense entry and analysis process. The user enters expense details, the application validates the amount and description, and a valid record is appended to the CSV file. The data is later grouped and summarized for display in tables and charts.

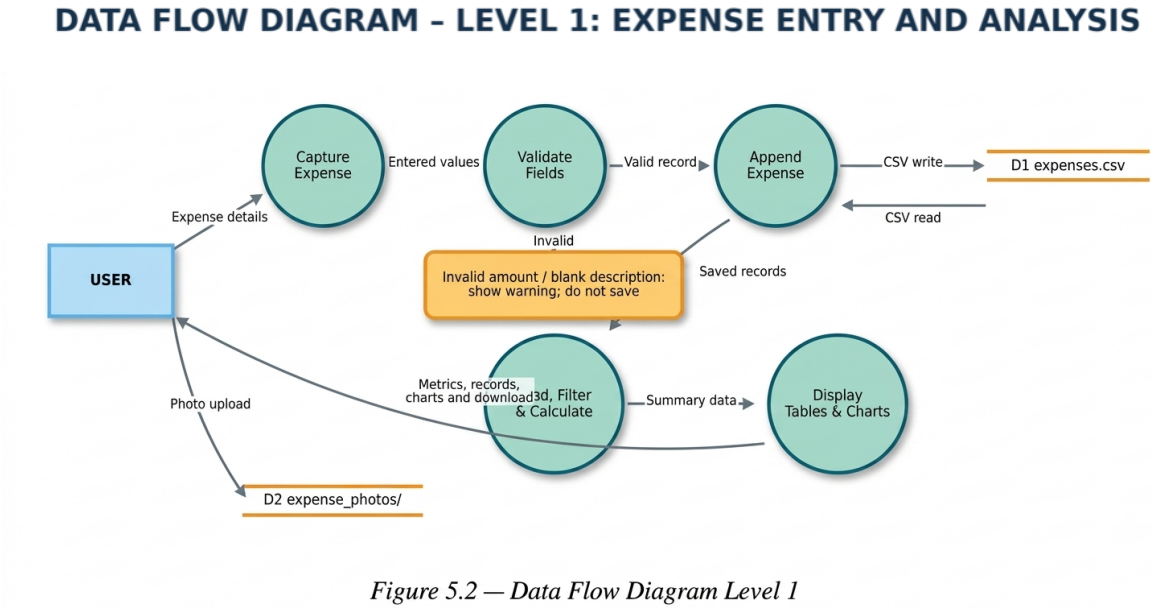


Figure 5.2 — Data Flow Diagram Level 1

5.3 Use Case Diagram

The application has one primary actor: the user. The user can add expenses, view records, filter and sort records, view summary metrics, analyze monthly and category-wise spending, view charts, download CSV data and upload a receipt or expense photo.

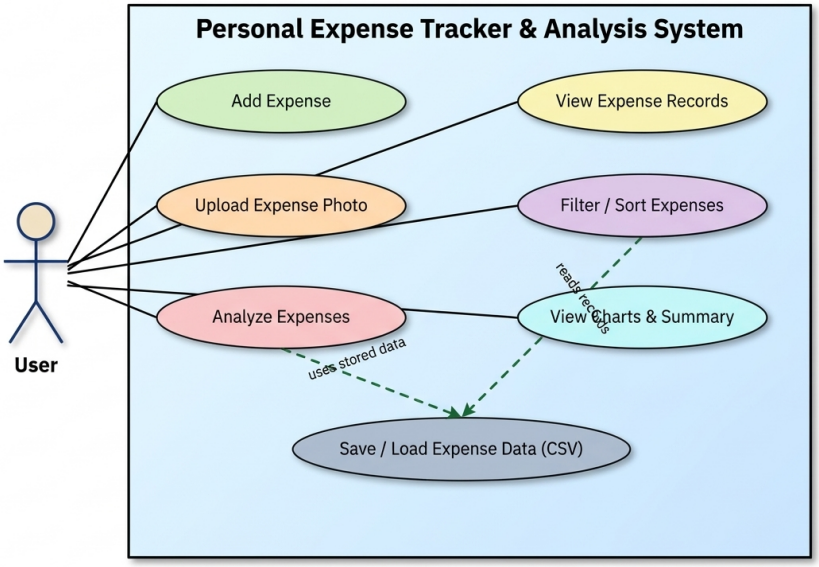


Figure 5.3 – Use Case Diagram

5.4 Entity–Relationship Diagram

The supplied CSV model contains one logical entity, Expense, with four attributes: Date, Category, Amount and Description. The project does not implement separate relational entities, primary-key fields or foreign-key relationships. Each row in expenses.csv represents one expense record.

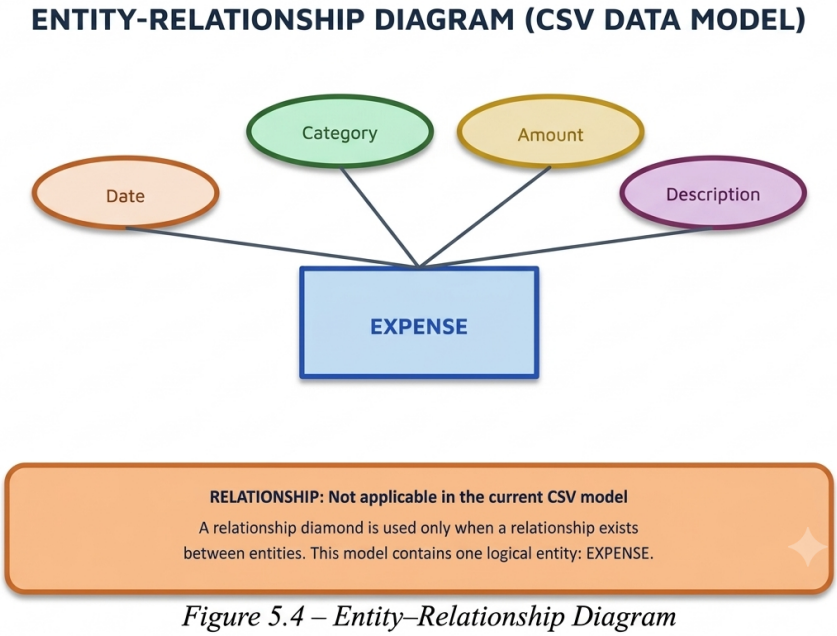


Figure 5.4 – Entity–Relationship Diagram

5.5 Activity Diagram

The activity begins when the user opens the application and enters expense details. After form submission, the application checks that the amount is greater than zero and that the description is not blank. If the checks pass, the record is added to the CSV file and the application refreshes the dashboard.

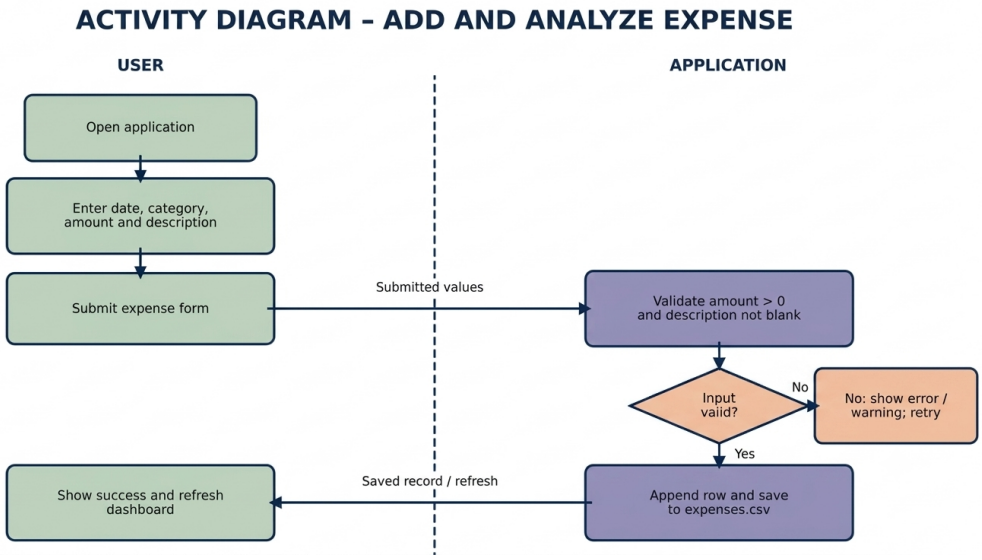


Figure 5.5 – Activity Diagram: Expense Entry

5.6 Flowchart

The flowchart describes the validation and saving logic used by the Add Expense form. Invalid values display an error or warning. Valid details are passed to the add_expense function, appended to the DataFrame and saved in expenses.csv.

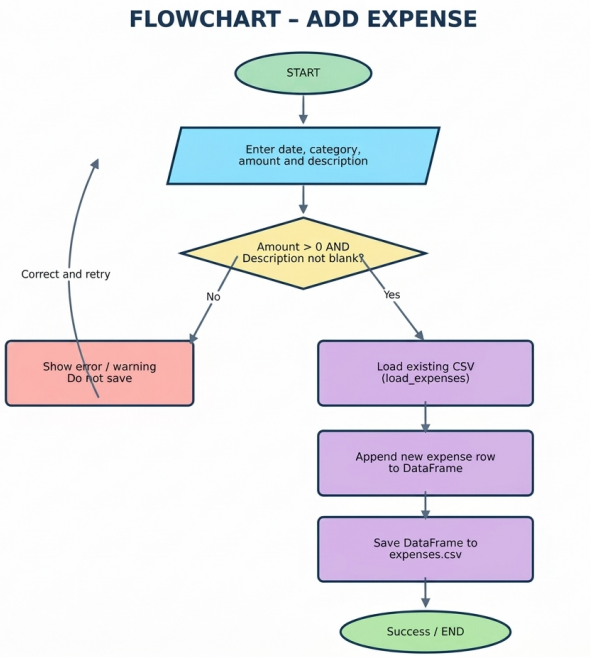


Figure 5.6 – Expense Entry Flowchart

CHAPTER 6

PROPOSED DESIGN

6.1 Design Approach

The application uses a simple single-file design. Streamlit controls the user interface and page execution. The load_expenses function reads and normalizes the CSV data, while save_expenses writes the updated DataFrame. The add_expense function creates a new row and appends it to the existing data. The remaining sections calculate summary values, display records, generate charts and handle downloads and photo uploads.

6.2 System Architecture Design

A user interacts with the application through a web browser. Streamlit renders the form, metric cards, data table, tabs and upload control. Python application logic validates form values and calls Pandas operations. Pandas reads and writes expenses.csv and prepares grouped data. Matplotlib generates the charts displayed in the browser. Uploaded receipt images are saved in the separate expense_photos directory.

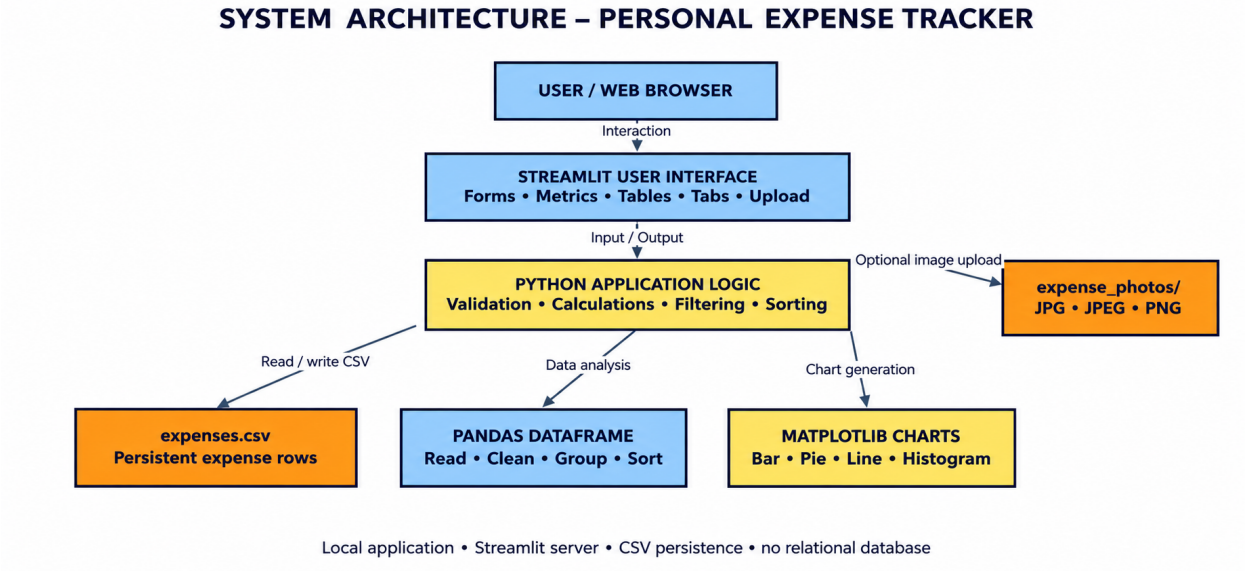


Figure 6.1 – System Architecture

6.3 User Interface (UI) Planning

- Application header: displays the Personal Expense Tracker title and a short subtitle.
- Add New Expense: date picker, category selection, amount input and description field with an Add Expense button.
- Summary cards: display Total, Average, Highest and Lowest values.
- Expense Records: shows a table with category filter and sorting options.
- Highest & Lowest Expenses: displays the amount, category and description of the extreme records.
- Spending Analysis: contains Monthly, Category-wise, Line Chart and Histogram tabs.
- CSV Download: provides a button to download the current records.
- Receipt / Expense Photo: accepts JPG, JPEG and PNG uploads and displays the uploaded image.

6.4 Data / Module Design

Source file / folder	Responsibility
app.py	Contains page configuration, custom CSS, expense loading and saving, entry form, metrics, table, analysis, download and image upload.
load_expenses()	Creates the CSV if missing, reads the file, ensures expected columns and converts date, amount, category and description values.
save_expenses(df)	Formats dates as YYYY-MM-DD and writes the DataFrame to expenses.csv.
add_expense(...)	Creates a new expense row, appends it to the loaded DataFrame and saves the result.
expenses.csv	Stores Date, Category, Amount and Description.
expense_photos/	Stores uploaded JPG, JPEG and PNG images.
requirements.txt	Lists Streamlit, Pandas and Matplotlib dependencies.

6.5 Design Considerations

The project uses a relative path based on the location of app.py, so the CSV and photo directory are referenced within the project folder. Amounts are converted to numeric values and invalid numeric values are filled with zero during loading. Dates are converted to datetime values. Empty data is handled by setting the summary metrics to zero and displaying a message instead of an expense table. The application is designed for local individual use rather than multi-user production deployment.

CHAPTER 7

EXPERIMENTAL RESULT

7.1 Overview of Experimental Setup

The supplied AB.zip package was inspected. The application source contains app.py, expenses.csv and requirements.txt, along with a bundled virtual environment and an empty expense_photos directory. The requirements file lists Streamlit, Pandas and Matplotlib. The report's numerical results are calculated from the six records present in the supplied expenses.csv file.

7.2 Expense Entry Results

The form accepts a date, one of the six predefined categories, an amount in rupees and a description. The amount field has a minimum value of ₹0.01. On submission, the code checks the amount and verifies that the trimmed description is not empty. A valid record is appended to the CSV file and a success message is displayed.

7.3 Summary and Record Management Results

The supplied CSV contains 6 records. The total recorded expenditure is ₹2,090.00, the average is ₹348.33, the highest single expense is ₹800.00, and the lowest single expense is ₹120.00. The category filter and sorting controls operate on a copy of the loaded DataFrame, so the display order does not change the saved CSV data.

Measure	Observed value from supplied CSV
Number of expense records	6
Total expenditure	₹2,090.00
Average expense	₹348.33
Highest expense	₹800.00
Lowest expense	₹120.00
Date range	2026-09-01 to 2026-10-31

7.4 Analytical Results

The supplied data records expenditure in Food, Travel, Education, Shopping and Entertainment. No record currently uses the Other category. The category totals and monthly totals below are calculated directly from expenses.csv.

Category	Total amount
Education	₹800.00
Shopping	₹500.00
Food	₹370.00
Entertainment	₹300.00
Travel	₹120.00

Month	Total amount
2026-09	₹1,970.00
2026-10	₹120.00

7.5 Graphical Analysis Results

The application creates five visual outputs. The monthly bar chart compares expenditure by calendar month. The category bar chart and pie chart compare category totals. The daily line chart displays the recorded amount for each date. The histogram shows how the expense amounts are distributed across value intervals.

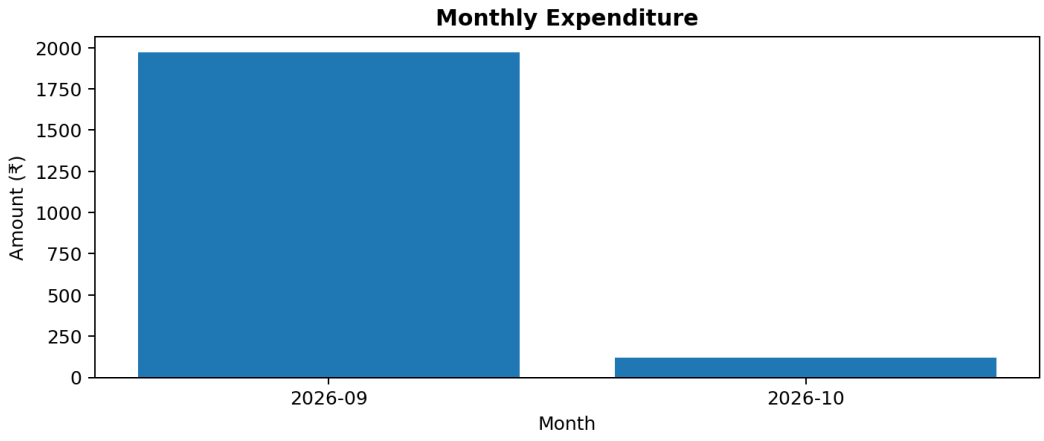


Figure 7.1 – Monthly Expenditure Chart

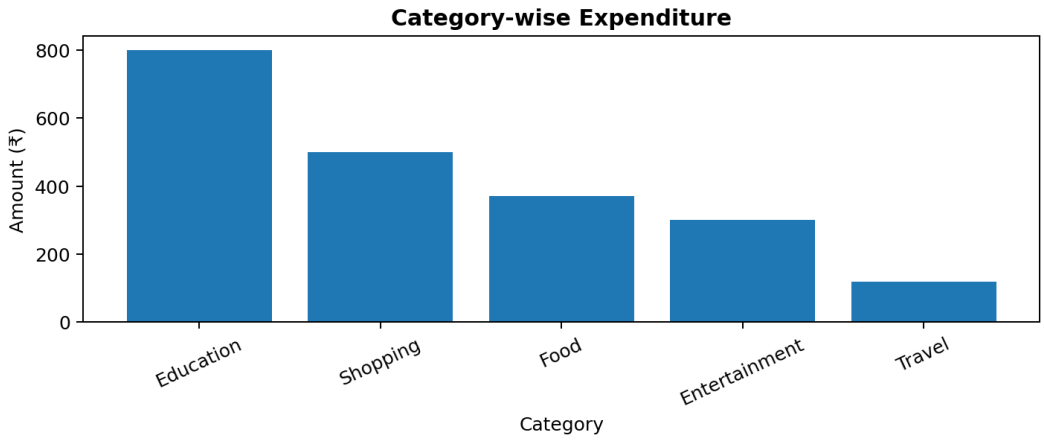


Figure 7.2 – Category-wise Expenditure Chart

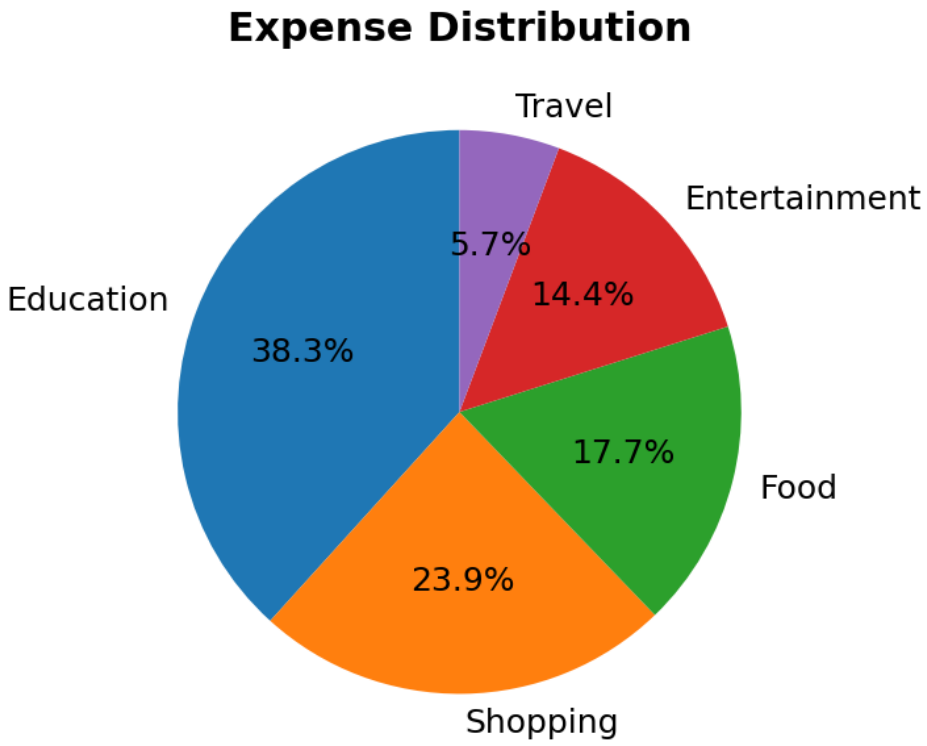


Figure 7.3 – Expense Distribution Pie Chart

7.5 Graphical Analysis Results (Continued)

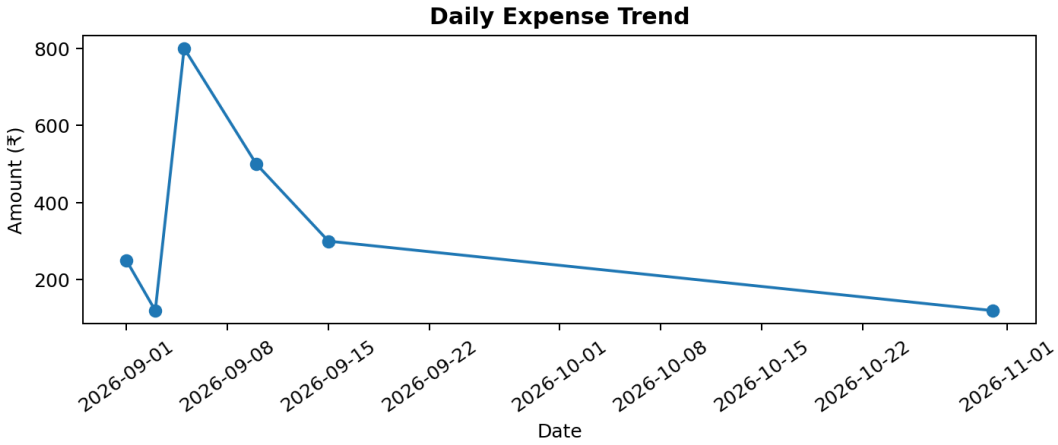


Figure 7.4 – Daily Expense Trend

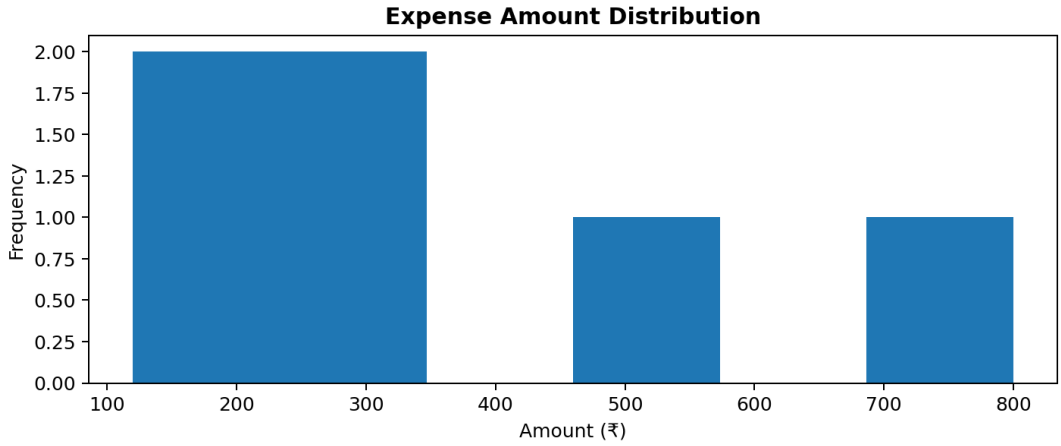


Figure 7.5 – Expense Amount Histogram

7.6 Testing Results

Test case	Expected result	Observation from source / sample
TC-01: Load existing CSV	Read stored expense records.	The loader reads expenses.csv and normalizes expected columns.
TC-02: Missing CSV	Create an empty CSV with expected columns.	The source checks file existence and creates the file when missing.
TC-03: Add valid expense	Append a new row and save CSV.	The form calls add_expense when validation succeeds.
TC-04: Amount is not positive	Show an error and do not save.	The form checks amount <= 0.
TC-05: Blank description	Show a warning and do not save.	The form checks description.strip().
TC-06: Category filter	Display only selected category.	The table view filters the DataFrame by Category.
TC-07: Sorting	Sort by date or amount.	Four sorting choices are implemented.
TC-08: CSV download	Download current records.	The download button is shown when records exist.
TC-09: Photo upload	Accept JPG/JPEG/PNG and save image.	The uploader restricts file types and writes bytes to expense_photos.
TC-10: Empty dataset	Show zero metrics and no-record message.	The source handles an empty DataFrame.

The project source was reviewed for implemented features. This documentation does not claim a live browser execution test. Before final academic submission, run the application on the target computer and capture actual screenshots of the running interface.

CHAPTER 8

FUTURE SCOPE

Edit and delete expenses: Allow users to correct an existing expense or remove an incorrect record.

Date-range filtering: Add start-date and end-date controls to review expenses for a selected period.

Budget management: Allow monthly or category-wise budgets and show warnings when spending approaches a limit.

User authentication: Add secure login and separate expense data for different users.

Database integration: Replace CSV storage with SQLite or MySQL when the number of records and concurrent users increase.

Receipt association: Store the uploaded photo path with a selected expense record so the receipt can be retrieved later.

Additional reports: Provide PDF and Excel reports with date-range and category summaries.

Recurring expenses: Support recurring entries such as rent, subscriptions and utility bills.

CHAPTER 9

CONCLUSION

The Personal Expense Tracker & Analysis System provides a simple way to record personal expenses and understand spending patterns through a browser-based dashboard. The application accepts expense details, stores records in a CSV file, displays summary metrics and supports category filtering and sorting. It also provides monthly and category-wise analysis, a daily expense trend, an amount histogram, CSV download and optional receipt/ photo upload.

The project demonstrates practical use of Python, Streamlit, Pandas and Matplotlib. The supplied code uses functions for loading, saving and adding expense records. It applies input checks to the amount and description and converts stored data into suitable date and numeric formats for analysis. The six sample records in the supplied CSV produce a total expenditure of ₹2,090.00 and an average expense of ₹348.33.

The current implementation is intentionally limited to a local, individual expense tracker. It does not include user authentication, record editing or deletion, budget alerts, a relational database or automatic association between uploaded photos and expense rows. These features can be considered in future development. Overall, the project meets its basic objective of recording, summarizing and visualizing personal expenses in a compact Python application.

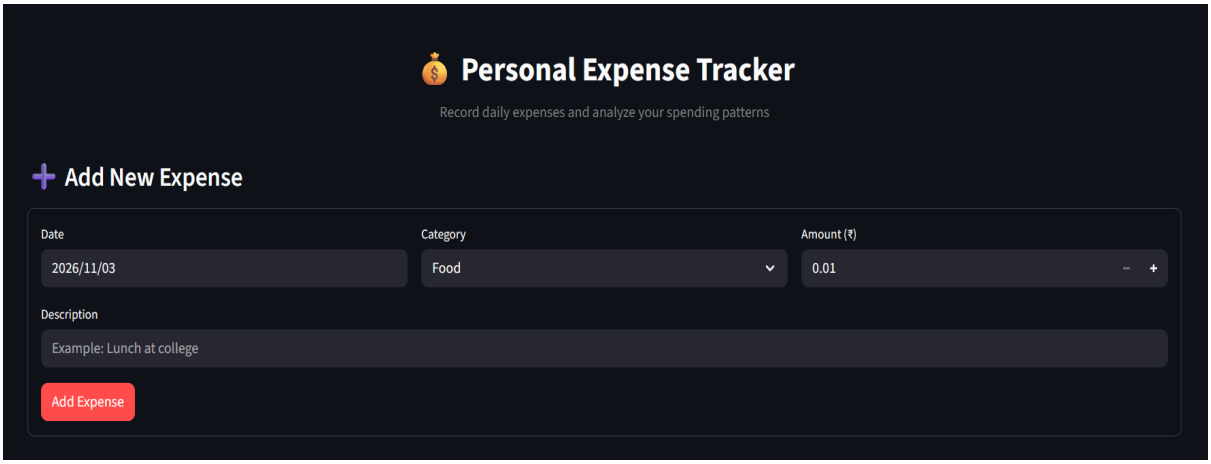
APPENDICES

APPENDIX A: APPLICATION INTERFACE

The supplied ZIP contains the application source code and sample CSV data but does not contain captured screenshots of the running browser interface. The following screen descriptions are based on the Streamlit components found in app.py. Insert genuine screenshots from the running application in the spaces below before final submission.

A.1 Application Header and Expense Entry

The page title and subtitle appear above the Add New Expense form. The form contains date, category, amount and description controls.



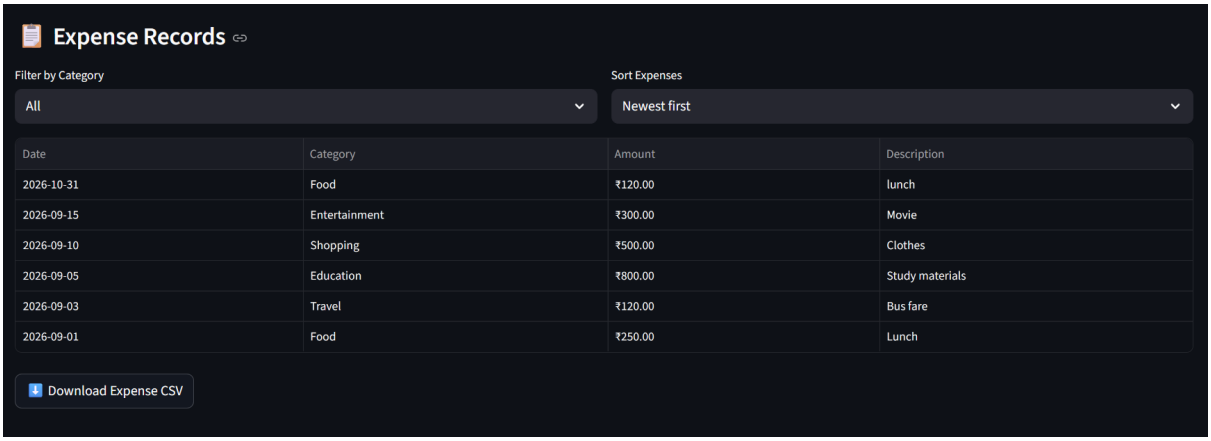
A.2 Summary Metrics

Four metric cards display Total, Average, Highest and Lowest expense values.



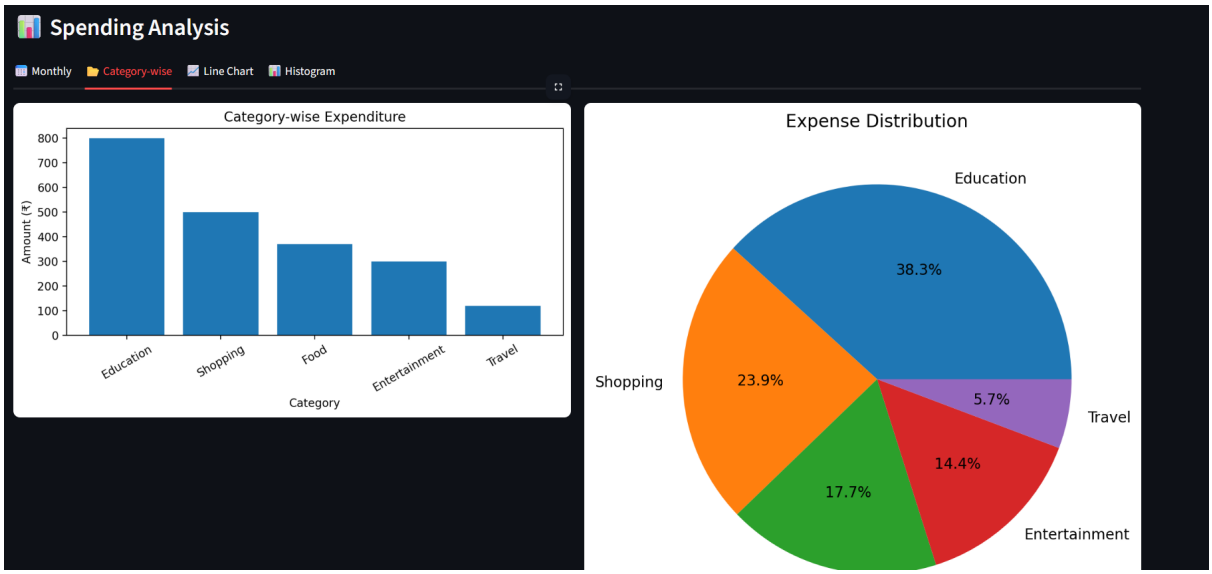
A.3 Expense Records

The records section contains category filtering, sorting options and a data table.



A.4 Spending Analysis

The analysis section contains Monthly, Category-wise, Line Chart and Histogram tabs.



A.5 Receipt / Expense Photo

The uploader accepts JPG, JPEG and PNG files and displays the selected image.



APPENDIX B: SAMPLE SOURCE CODE

B.1 Expense CSV Columns

The application expects four columns in expenses.csv:

```
Date,Category,Amount,Description
```

B.2 Loading Expense Data

```
def load_expenses():
    if not CSV_FILE.exists():
        df = pd.DataFrame(columns=COLUMNS)
        df.to_csv(CSV_FILE, index=False)
    try:
        df = pd.read_csv(CSV_FILE)
    except Exception:
        df = pd.DataFrame(columns=COLUMNS)
    for column in COLUMNS:
        if column not in df.columns:
            df[column] = ""
   df = df[COLUMNS]
```

B.3 Adding an Expense

```
def add_expense(expense_date, category, amount, description):
    df = load_expenses()
    new_expense = pd.DataFrame(
        [[pd.Timestamp(expense_date), category,
          float(amount), description]],
        columns=COLUMNS
    )
    df = pd.concat([df, new_expense], ignore_index=True)
    save_expenses(df)
```

B.4 Summary Calculations

```
total = df["Amount"].sum()
average = df["Amount"].mean()
highest = df["Amount"].max()
lowest = df["Amount"].min()
```

APPENDIX C: CSV DATA STRUCTURE

The supplied expenses.csv file contains six sample records. Its schema is shown below.

Column	Example value	Purpose
Date	2026-09-01	Expense date in YYYY-MM-DD format.
Category	Food	Predefined spending category.
Amount	250.0	Numeric expense value in rupees.
Description	Lunch	Short description of the transaction.

C.1 Sample Data Summary

Date	Category	Amount (₹)	Description
2026-09-01	Food	250.00	Lunch
2026-09-03	Travel	120.00	Bus fare
2026-09-05	Education	800.00	Study materials
2026-09-10	Shopping	500.00	Clothes
2026-09-15	Entertainment	300.00	Movie
2026-10-31	Food	120.00	lunch

APPENDIX D: TEST CASES

The following cases are based on the validation and user-interface logic present in the supplied source code. Actual execution outcomes should be confirmed on the deployment computer.

ID	Input / action	Expected behaviour
D-01	Enter amount greater than zero and description.	Save the expense and show success message.
D-02	Submit amount equal to zero.	Show amount validation error.
D-03	Submit blank description.	Show warning and do not save.
D-04	Choose a category filter.	Show matching category rows.
D-05	Choose Newest first.	Sort records by date descending.
D-06	Choose Highest amount.	Sort records by amount descending.
D-07	Click CSV download when records exist.	Download expenses.csv.
D-08	Upload JPG, JPEG or PNG.	Save and display the uploaded image.
D-09	Upload unsupported file type.	File type should be rejected by uploader.
D-10	Open application with no records.	Display zero metrics and no-record message.

REFERENCES

- **Python Software Foundation, Python Documentation:**

  https://docs.python.org/3/

- **Streamlit Documentation:**

  https://docs.streamlit.io/

- **Pandas Documentatio:**

  https://pandas.pydata.org/docs/

- **Matplotlib Documentation:**

  https://matplotlib.org/stable/

- **Brainware University, Department of Computational Sciences** – academic project report format supplied as the reference Word document.